Lecture 07 (B06): Brock–Mirman with DEQNs — Deterministic Case

Neural network architecture, supervised vs unsupervised, deterministic Brock–Mirman

Simon Scheidegger

HEC, University of Lausanne

What is a Deep Neural Network?

A deep neural network (DNN) $\mathcal{N}_\rho : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ is a parametric function approximator.

Universal approximation (Cybenko 1989, Hornik et al. 1989):

$$\forall f \in C(\mathcal{K}, \mathbb{R}),\ \forall \varepsilon > 0,\ \ \exists\, \rho^\star : \quad \sup_{\boldsymbol{x} \in \mathcal{K}} \left\| \mathcal{N}_{\rho^\star}(\boldsymbol{x}) - f(\boldsymbol{x}) \right\| < \varepsilon$$

($\mathcal{K} \subset \mathbb{R}^{d_{\text{in}}}$ compact; the norm is the supremum over $\mathcal{K}$.)

In economics:

- Input: state variables $\boldsymbol{x}_t \in \mathbb{R}^d$ (capital, productivity, wealth distribution, ...)
- Output: policy variables (consumption, savings, prices, ...)
- Parameters ρ: weights and biases, found by training

DNN Layer Structure

$$\boxed{\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}}}} \rightarrow \boxed{\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}} \rightarrow \boxed{\boldsymbol{W}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)}} \rightarrow \cdots \rightarrow \boxed{\boldsymbol{W}^{(L)}\boldsymbol{a}^{(L-1)} + \boldsymbol{b}^{(L)}}$$

Input | Hidden 1 | Hidden 2 | Output

Each layer performs an affine transformation:

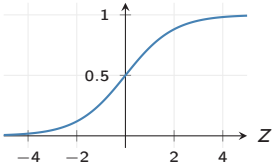
$$\boldsymbol{z}^{(\ell)} = \boldsymbol{W}^{(\ell)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \qquad \ell = 1, \ldots, L$$

where $\boldsymbol{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ and $\boldsymbol{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$.

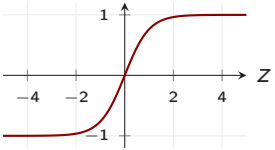
Without nonlinear activations, the entire network collapses to a single affine map.

Activation Functions

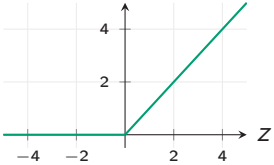
Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$



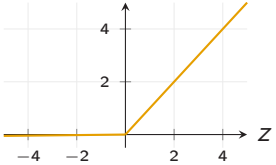
Tanh: $\tanh(z)$



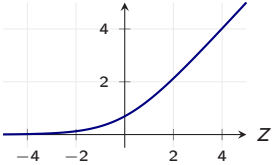
ReLU: $\max(0, z)$



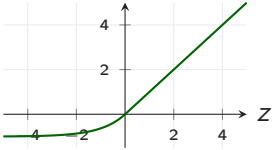
Leaky ReLU: $\max(0.01z, z)$



Softplus: $\ln(1 + e^z)$



ELU: z **if** $z \geq 0$; $e^z - 1$ **if** $z < 0$



DNN with Nonlinear Activations

$$\boxed{\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}}}} \rightarrow \boxed{\phi^{(1)}\big(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\big)} \rightarrow \boxed{\phi^{(2)}\big(\boldsymbol{W}^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)}\big)} \rightarrow \boxed{\boldsymbol{W}^{(L)}\boldsymbol{a}^{(L-1)} + \boldsymbol{b}^{(L)}}$$

Input | Hidden 1 | Hidden 2 | Output (no activation)

Each hidden layer applies a nonlinear activation $\phi^{(\ell)}$:

$$\boldsymbol{a}^{(\ell)} = \phi^{(\ell)}\big(\boldsymbol{W}^{(\ell)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}\big), \qquad \ell = 1, \ldots, L-1$$

- The output layer is typically linear (for regression) or uses softplus (to enforce positivity, e.g. consumption)
- The full network: $\mathcal{N}_\rho(\boldsymbol{x}) = \boldsymbol{W}^{(L)}\boldsymbol{a}^{(L-1)} + \boldsymbol{b}^{(L)}$

Finding Parameters: Supervised Learning

Standard approach (when labeled data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ are available):

1. **Collect labeled data:** $\boldsymbol{x}_i \xrightarrow{\text{true mapping}} y_i$
2. **Define a loss function:**

$$\ell_\rho^{\text{sup}} = \frac{1}{N}\sum_{i=1}^{N}\left\|y_i - \mathcal{N}_\rho(\boldsymbol{x}_i)\right\|^2 \qquad \text{(mean squared error)}$$

3. **Update via SGD** (η = learning rate; the symbol α is reserved for the Cobb–Douglas capital share later in the deck):

$$\rho^{\text{new}} = \rho^{\text{old}} - \eta \cdot \left.\frac{\partial \ell_\rho}{\partial \rho}\right|_{\rho=\rho^{\text{old}}}$$

This is the recipe from Lecture 1. But in economics, we rarely have labeled data...

The "Data Problem" in Economics

Deep learning excels when massive labeled datasets are available.

But in computational economics:

- We solve functional equations, not classification/regression tasks
- There are no labeled pairs $(\boldsymbol{x}_i, y_i)$
- Traditional approach: time iteration on a grid, but grids do not scale

Key insight:

- We know the structural equations $G(\cdot) = 0$
- We can check how well a candidate policy satisfies them
- $\Rightarrow$ Use the equilibrium error as the loss function!

Traditional Approach: Time Iteration on a Grid

Algorithm: Grid-Based Collocation

Input: Grid $\mathcal{G} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_M\} \subset \mathbb{R}^d$, initial guess $p^{(0)}$
for $k = 0, 1, 2, \ldots$ until convergence **do**
  for each grid point $\boldsymbol{x}_i \in \mathcal{G}$ **do**
    Solve $G\big(\boldsymbol{x}_i,\, p^{(k)}(\boldsymbol{x}_i),\, p^{(k)}\big) = 0$ for $p^{(k+1)}(\boldsymbol{x}_i)$
  end for
  Interpolate $p^{(k+1)}$ between grid points
end for

Problems:

- Cartesian grids need $M = n^d$ points, infeasible beyond $d \sim 4$–5 at sensible resolution (10^5 at $d = 5$, 10^{10} at $d = 10$)
- Sparse grids (Smolyak, adaptive) extend the practical reach only to $d \sim 20$–50
- Heterogeneous-agent or many-country DSGE models routinely need $d \gg 50$, beyond any grid-based method

The DEQN Loss Function

Key idea: replace labeled data with economic equilibrium conditions.

Let $G(\boldsymbol{x}, f(\boldsymbol{x})) = 0$ be the system of equilibrium equations. Define:

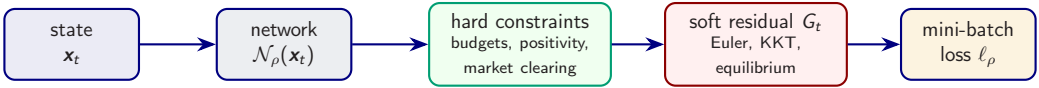
$$\boxed{\ell_\rho = \frac{1}{N}\sum_{i=1}^{N}\left\| G\left(\boldsymbol{x}_i, \mathcal{N}_\rho(\boldsymbol{x}_i)\right)\right\|^2}$$

- $\mathcal{N}_\rho(\boldsymbol{x}_i)$ is the neural net's predicted policy at state $\boldsymbol{x}_i$
- $G(\cdot)$ measures the equilibrium error (e.g. Euler equation residual)
- Small ℓ_ρ means the policy nearly satisfies the equations on the sampled states

$\Rightarrow$ No labels needed, so this is **unsupervised** (or "self-supervised") learning.

In stochastic models, expectations inside G are approximated, e.g. by path averaging or quadrature.

Inside One DEQN Training Step



- Encode what can be enforced exactly in the architecture.
- Minimize only the genuinely non-closed-form equilibrium conditions.
- Draw states from the model's own simulation, so training focuses on the ergodic set.

Training Algorithm for DEQNs

Algorithm 1: Deep Equilibrium Net Training

Input: Initial state $\boldsymbol{x}_0$, network $\mathcal{N}_\rho$, learning rate η, episodes E, simulation horizon T_{sim}, training steps T_{train}

for episode $e = 1, \ldots, E$ **do**

  Simulate path: $\boldsymbol{x}_0 \to \boldsymbol{x}_1 \to \cdots \to \boldsymbol{x}_{T_{\text{sim}}}$ using $\mathcal{N}_\rho$ and transition law

  for gradient step $t = 1, \ldots, T_{\text{train}}$ **do**

    Draw mini-batch $\mathcal{B} \subset \{\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{T_{\text{sim}}}\}$

    Compute loss: $\ell_\rho = \frac{1}{|\mathcal{B}|} \sum_{\boldsymbol{x}_i \in \mathcal{B}} \| G(\boldsymbol{x}_i, \mathcal{N}_\rho(\boldsymbol{x}_i)) \|^2$

    Update: $\rho \leftarrow \rho - \eta \cdot \nabla_\rho \ell_\rho$

  end for

end for

Output: Trained network $\mathcal{N}_{\rho^\star}$ approximating the policy function

Note: Training points come from the model's own simulation, so the network learns on the ergodic distribution, focusing on economically relevant regions.

Supervised vs. Unsupervised (DEQN) Learning

Supervised

Labeled data $(\boldsymbol{x}_i, y_i)$

$\downarrow$

$\ell = \|y_i - \mathcal{N}_\rho(\boldsymbol{x}_i)\|^2$

$\downarrow$

SGD $\rightarrow \rho^\star$

DEQN (Unsupervised)

Equations $G(\boldsymbol{x}, p(\boldsymbol{x})) = 0$

$\downarrow$

$\ell = \|G(\boldsymbol{x}_i, \mathcal{N}_\rho(\boldsymbol{x}_i))\|^2$

$\downarrow$

SGD $\rightarrow \rho^\star$

Key difference: DEQNs replace labeled data with structural economic equations.

DEQN: Key Recap

Replace Labels

Instead of labeled data $(\boldsymbol{x}_i, y_i)$, use the economic equilibrium equations $G(\cdot) = 0$ as the loss.

Replace Grids

Instead of n^d grid points, use a neural network $\mathcal{N}_\rho$ that maps states to policies globally.

Replace TI

Instead of time iteration with interpolation, use SGD on simulated paths from the model.

Result: A grid-free, simulation-based method that can handle much higher-dimensional models than standard grids and maps naturally to GPUs.

Part III

Brock–Mirman Benchmark

Brock–Mirman (1972): Setup

Social planner's problem. Choose a consumption sequence to maximize expected lifetime log utility:

$$\max_{\{C_t\}_{t=0}^{\infty}} \mathbb{E}\left[\sum_{t=0}^{\infty} \beta^t \ln C_t\right]$$

Resource constraint (full depreciation, $\delta = 1$):

$$C_t + K_{t+1} \;=\; z_t K_t^{\alpha}, \qquad K_0 > 0 \text{ given.}$$

Stochastic productivity. TFP z_t follows an AR(1) in logs:

$$\ln z_{t+1} \;=\; \varrho \ln z_t + \sigma_z \, \epsilon_{t+1}, \qquad \epsilon_{t+1} \sim \mathcal{N}(0,1).$$

State & control. State $x_t = (K_t, z_t)$; control C_t. Capital then follows from the budget as $K_{t+1} = z_t K_t^{\alpha} - C_t$.

Next two slides: derive the Euler equation from first principles, then obtain the closed-form policy – the benchmark we will later compare the DEQN solution to.

Deriving the Euler Equation (Lagrangian)

Step 1. Lagrangian. Attach a multiplier $\lambda_t \geq 0$ to each period's budget constraint:

$$\mathcal{L} \;=\; \mathbb{E}\left[\sum_{t=0}^{\infty} \beta^t \Big[\, \ln C_t \;-\; \lambda_t \left(C_t + K_{t+1} - z_t K_t^{\alpha}\right)\Big]\right].$$

Step 2. Take partial derivatives with respect to the choices at date t:

Consumption FOC $\partial\mathcal{L}/\partial C_t = 0$:

$$\beta^t\left(\frac{1}{C_t} - \lambda_t\right) = 0 \implies \lambda_t = \frac{1}{C_t}.$$

Capital FOC $\partial\mathcal{L}/\partial K_{t+1} = 0$:

$$-\beta^t \lambda_t \;+\; \beta^{t+1}\,\mathbb{E}\big[\lambda_{t+1}\,\alpha z_{t+1} K_{t+1}^{\alpha-1}\big] = 0.$$

Step 3. Eliminate λ. Substitute $\lambda_t = 1/C_t$, $\lambda_{t+1} = 1/C_{t+1}$ and divide by β^t:

$$\boxed{\frac{1}{C_t} \;=\; \beta\,\mathbb{E}\left[\frac{\alpha\, z_{t+1}\, K_{t+1}^{\alpha-1}}{C_{t+1}}\right]} \qquad \text{(stochastic Euler equation)}$$

Reading: marginal utility of today's consumption $=$ discounted expected marginal utility of tomorrow's consumption $\times$ gross marginal product of capital at $t{+}1$. The transversality condition $\lim_{t\to\infty} \mathbb{E}[\beta^t \lambda_t K_{t+1}] = 0$ rules out bubbles.

Closed-Form Policy via Guess-and-Verify

Guess. Consumption is a constant fraction $1-\phi$ of output:

$$C_t \;=\; (1-\phi)\, z_t K_t^{\alpha}, \qquad \phi \in (0,1) \text{ to be determined.}$$

The budget then implies $K_{t+1} = \phi\, z_t K_t^{\alpha}$.

Substitute into the Euler equation. With $C_{t+1} = (1-\phi)\, z_{t+1} K_{t+1}^{\alpha}$,

$$\frac{\alpha\, z_{t+1}\, K_{t+1}^{\alpha-1}}{C_{t+1}} \;=\; \frac{\alpha\, z_{t+1}\, K_{t+1}^{\alpha-1}}{(1-\phi)\, z_{t+1}\, K_{t+1}^{\alpha}} \;=\; \frac{\alpha}{(1-\phi)\, K_{t+1}}.$$

Note: z_{t+1} cancels, so the conditional expectation collapses (log + Cobb–Douglas + full depreciation).

Solve for ϕ. Plug $K_{t+1} = \phi z_t K_t^{\alpha}$ on both sides of the Euler:

$$\underbrace{\frac{1}{(1-\phi)\, z_t K_t^{\alpha}}}_{\text{LHS}} \;=\; \beta \cdot \underbrace{\frac{\alpha}{(1-\phi)\, \phi\, z_t K_t^{\alpha}}}_{\text{RHS}} \;\Longrightarrow\; 1 = \frac{\beta\alpha}{\phi} \;\Longrightarrow\; \phi = \beta\alpha.$$

Closed-form policy.

$$\boxed{C_t \;=\; (1-\beta\alpha)\, z_t K_t^{\alpha}, \qquad K_{t+1} \;=\; \beta\alpha\, z_t K_t^{\alpha}}$$

This benchmark provides an analytic target against which we will measure the DEQN's trained policy.